

Sparse Ensembles: Do Routing Mechanisms Benefit from Ensemble Effects?

Anonymous authors

Paper under double-blind review

Abstract

Ensemble effects among predictive models have been studied extensively and are a well-established source of improved generalization. In sparse ensembles, however, prediction is only one part of the problem: inputs must also be routed to the appropriate models. Whether this routing mechanism itself benefits from ensemble effects remains largely unexplored. We provide a systematic study of ensembling the routing mechanism, examining when increasing the number of routers improves performance, how their outputs should be combined, and whether explicitly encouraging diversity among routers is beneficial. Across classification and regression tasks, we characterize how these choices interact with the size and capacity of the underlying models and the routing ensemble.

1 Introduction

Ensemble methods are a well-established class of machine learning techniques that improve generalization by aggregating the predictions of multiple models. Their effectiveness is typically attributed to two complementary factors: the performance of the individual predictors and the diversity among them. Improving individual predictors reduces bias, while diversity promotes error de-correlation, allowing aggregation to cancel out model-specific mistakes.

Classical ensemble methods induce diversity implicitly via bagging, such as random forests, where predictors are trained on different subsets of the data, whereas boosting methods, such as LightGBM, construct predictors sequentially to focus on errors made by earlier models. In deep learning, diversity can be encouraged explicitly through mechanisms such as negative-correlation learning (Chen & Yao, 2009) or expert orthogonalization (Liu et al., 2024). However, Fort et al. (2019) showed that, in many cases, initializing experts with different random weights is sufficient to induce substantial diversity.

Sparse ensembles, in which only a subset of predictors is active for a given input, have gained prominence because of their computational advantages in large-scale language modeling. In the most prominent setting, each expert, or subset of experts, is responsible for a particular region of the input domain, allowing the ensemble to approximate the capacity of a larger model while retaining the computational cost of a smaller one. Existing variants balance computation and ensemble effects in different ways: selecting the top- k predictors for each input (Shazeer et al., 2017), maximizing cost reduction by routing each input to a single predictor (Fedus et al., 2022), or allocating a dynamic number of predictors per input (Wang et al., 2025). Routing-based methods decompose prediction into two coupled subproblems: a *delegation problem*, handled by a *router* or *delegator*, and a *prediction problem*, handled by the *experts* or *predictors*.

While ensembles typically focus on improving the predictive capacity of the experts, the delegation mechanism itself may also benefit from ensembling. This motivates studying whether increasing the number of delegators improves the allocation of unseen samples to predictors and, consequently, the overall performance of the ensemble. Despite recent methods that combine multiple routers (Zhang et al., 2025; Ran et al., 2025), the effect of scaling the number of delegators remains underexplored. In particular, to our knowledge, there is no systematic analysis of this setting through the lens of ensemble theory.

Contributions In this work we provide a systematic study of scaling the number of delegators in image classification tasks, focusing on CIFAR10 (Krizhevsky, 2009) and SVHN (Netzer et al., 2011), and tabular regression tasks, Bike sharing dataset (Fanaee-T, 2013) and Energy production dataset (Zaki, 2026). Specifically this work systematically tests scaling of number of delegators, number of predictors, delegator widths, and predictor widths. Furthermore, two aggregation methods are tested, probabilities product and probabilities sum. We find:

- **(C1) When scaling number of delegators helps:** Under which scenarios
- **(C2) What aggregation method is better for the delegators:** Is sum better?
- **(C3) Ambiguity gradient control:** We examine how separately controlling predictor and delegator ambiguity gradients affects ensemble performance.
- **(C4) Predictor and delegator decompositions:** We formulate a performance–ambiguity decomposition for delegators through the predictions their weights induce, and use a restricted oracle to separate residual predictor loss from recoverable delegation loss.

2 Background

We consider a supervised learning setting with a training dataset $\mathbb{D} = \{(\mathbf{x}_n, \mathbf{y}_n)\}_{n=1}^N$ of N i.i.d. samples, where $\mathbf{x} \in \mathbb{R}^{d_x}$ denotes the input and $\mathbf{y} \in \mathbb{R}^{d_y}$ the corresponding real-valued output for regression, or $\mathbf{y} \in \{0, 1\}^{d_y}$ a one-hot target for classification. We write $\Delta_M = \{\mathbf{a} \in \mathbb{R}^M : a_m \geq 0, \sum_{m=1}^M a_m = 1\}$ for the probability simplex.

We define an ensemble of M predictors $f_m = f_{\theta_m}$, with parameters $\Theta = (\theta_1, \dots, \theta_M)$, and an ensemble of L delegators $g_l = g_{\phi_l}$, with parameters $\Phi = (\phi_1, \dots, \phi_L)$. A predictor produces an output vector $\hat{\mathbf{y}}_m(\mathbf{x}) = f_m(\mathbf{x})$, which is a probability vector for classification. A delegator produces a weight proposal $\mathbf{a}_l(\mathbf{x}) = g_l(\mathbf{x}) \in \Delta_M$. Thus, predictors estimate the target, while delegators determine how predictors are combined.

For any weights $\mathbf{a} \in \Delta_M$, define the predictor aggregate as prescribed in Wood et al. (2023):

$$F(\mathbf{x}; \mathbf{a}) = \begin{cases} \sum_{m=1}^M a_m \hat{\mathbf{y}}_m(\mathbf{x}), & \text{regression,} \\ \frac{\bigodot_{m=1}^M \hat{\mathbf{y}}_m(\mathbf{x})^{a_m}}{\sum_{c=1}^{d_y} \prod_{m=1}^M \hat{y}_{m,c}(\mathbf{x})^{a_m}}, & \text{classification.} \end{cases} \quad (1)$$

Here $\bigodot$ denotes componentwise multiplication and $\mathbf{v}^{\odot \mathbf{a}}$ denotes componentwise exponentiation. The delegator proposals are combined into $\bar{\mathbf{a}}(\mathbf{x}) \in \Delta_M$ using either an arithmetic or normalized geometric mean (Section 4.2). For an input $\mathbf{x}$, the final prediction is $\hat{\mathbf{y}} = F(\mathbf{x}; \bar{\mathbf{a}}(\mathbf{x}))$, and the prediction induced by delegator l alone is $\hat{\mathbf{y}}^{(l)} = F(\mathbf{x}; \mathbf{a}_l(\mathbf{x}))$. With no delegators ($L = 0$), we use uniform weights $\bar{a}_m = 1/M$.

2.1 Predictor performance and ambiguity

For a fixed sample $(\mathbf{x}, \mathbf{y})$, write $\hat{\mathbf{y}}_m = f_m(\mathbf{x})$ for each predictor’s output. Let $\text{SE}(\hat{\mathbf{y}}, \mathbf{y}) = d_y^{-1} \|\hat{\mathbf{y}} - \mathbf{y}\|_2^2$ and $\text{CE}(\hat{\mathbf{y}}, \mathbf{y}) = -\sum_{c=1}^{d_y} y_c \log \hat{y}_c$ denote squared error and cross-entropy. For any weights $\mathbf{a} \in \Delta_M$, the classical squared-error decomposition (Krogh & Vedelsby, 1994) is

$$\text{SE}(F(\mathbf{x}; \mathbf{a}), \mathbf{y}) = \underbrace{\sum_{m=1}^M a_m \text{SE}(\hat{\mathbf{y}}_m, \mathbf{y})}_{\text{weighted predictor loss } \mathcal{E}_P(\mathbf{a})} - \underbrace{\sum_{m=1}^M a_m \text{SE}(F(\mathbf{x}; \mathbf{a}), \hat{\mathbf{y}}_m)}_{\text{weighted predictor ambiguity } \mathcal{A}_P(\mathbf{a})}. \quad (2)$$

For cross-entropy with geometric aggregation, the corresponding decomposition is (Wood et al., 2023)

$$\text{CE}(F(\mathbf{x}; \mathbf{a}), \mathbf{y}) = \underbrace{\sum_{m=1}^M a_m \text{CE}(\hat{\mathbf{y}}_m, \mathbf{y})}_{\text{weighted predictor loss } \mathcal{E}_P(\mathbf{a})} - \underbrace{\sum_{m=1}^M a_m \text{KL}(F(\mathbf{x}; \mathbf{a}) \parallel \hat{\mathbf{y}}_m)}_{\text{weighted predictor ambiguity } \mathcal{A}_P(\mathbf{a})}. \quad (3)$$

In both cases, $\mathcal{E}_P$ measures individual predictor accuracy, while $\mathcal{A}_P$ measures disagreement with the aggregate output. Lower individual loss and greater ambiguity at fixed individual loss improve ensemble performance. We use ℓ for the task loss (SE or CE) and d for its ambiguity measure (SE or KL), so both identities read $\ell(F(\mathbf{x}; \mathbf{a}), \mathbf{y}) = \mathcal{E}_P(\mathbf{a}) - \mathcal{A}_P(\mathbf{a})$.

Explicit maximization of ambiguity has been shown to be potentially harmful, since it can increase the bias and variance of the individual models, leading to higher generalization error (Ortega et al., 2022; Wood et al., 2023). In contrast, prior work indicates that using different random initializations is often sufficient to induce test-time diversity among ensemble members (Fort et al., 2019; Ortega et al., 2022). This motivates controlling ambiguity gradients separately from the loss decomposition itself (Section 4.3).

3 Delegator decompositions

3.1 Delegator performance and ambiguity

There is no target weight vector for a delegator. Instead, we assess each proposal through the predictor ensemble it induces. The loss $\ell(\hat{\mathbf{y}}^{(l)}, \mathbf{y})$ measures whether delegator l proposes weights that yield an accurate prediction. Its ambiguity $d(\hat{\mathbf{y}}, \hat{\mathbf{y}}^{(l)})$ measures disagreement with the final prediction, rather than disagreement between weight vectors themselves.

For arithmetic aggregation $\bar{\mathbf{a}} = L^{-1} \sum_{l=1}^L \mathbf{a}_l$, the final prediction is also the appropriate equal-weight aggregate of the induced predictions: their arithmetic mean for regression and their normalized geometric mean for classification. Applying the classical ambiguity decomposition (Equations 2–3) to these predictions gives

$$\ell(\hat{\mathbf{y}}, \mathbf{y}) = \underbrace{\frac{1}{L} \sum_{l=1}^L \ell(\hat{\mathbf{y}}^{(l)}, \mathbf{y})}_{\text{individual delegator loss } \mathcal{E}_D} - \underbrace{\frac{1}{L} \sum_{l=1}^L d(\hat{\mathbf{y}}, \hat{\mathbf{y}}^{(l)})}_{\text{delegator ambiguity } \mathcal{A}_D}. \quad (4)$$

The derivation for both tasks is given in Appendix A.2. Thus, each delegator should propose weights yielding a good predictor ensemble, while diversity is measured in the outputs these proposals induce. A larger $\mathcal{A}_D$ is beneficial at fixed $\mathcal{E}_D$. Each delegator’s loss also has the predictor decomposition

$$\ell(\hat{\mathbf{y}}^{(l)}, \mathbf{y}) = \mathcal{E}_P(\mathbf{a}_l) - \mathcal{A}_P(\mathbf{a}_l). \quad (5)$$

Equation 4 does not generally hold for geometric aggregation of delegation weights. The two terms remain defined, but their difference need not equal the final loss. Both terms require $L \geq 1$.

3.2 Delegator error

We also separate the final loss into a predictor loss under reference weights and a delegation regret. This is distinct from the performance–ambiguity decompositions above. For fixed predictor outputs and a sample $(\mathbf{x}, \mathbf{y})$, define the unrestricted oracle weights by

$$\mathbf{a}^*(\mathbf{x}, \mathbf{y}) \in \arg \min_{\mathbf{a} \in \Delta_M} \ell(F(\mathbf{x}; \mathbf{a}), \mathbf{y}). \quad (6)$$

The loss decomposes exactly as

$$\ell(\hat{\mathbf{y}}, \mathbf{y}) = \underbrace{\ell(F(\mathbf{x}; \mathbf{a}^*), \mathbf{y})}_{\text{predictor loss under optimal weights}} + \underbrace{[\ell(\hat{\mathbf{y}}, \mathbf{y}) - \ell(F(\mathbf{x}; \mathbf{a}^*), \mathbf{y})]}_{\text{delegator regret}}. \quad (7)$$

The first term is the minimum loss attainable by any admissible delegation weights for the given predictor outputs; the second is nonnegative. For regression, the first term is the squared distance to the convex hull of $\{\hat{\mathbf{y}}_m\}_{m=1}^M$, divided by d_y , and is zero exactly when the target lies in that hull. This convex-hull interpretation does not apply to geometrically aggregated classification probabilities.

The unrestricted, label-dependent oracle can be overly optimistic: with sufficiently many predictors, arbitrary sample-specific weights may interpolate regression targets. We therefore use a restricted delegator oracle with the same architecture as the learned delegators. With predictors fixed, this oracle seeks weights that reduce the ensemble loss. Its aggregate weights are denoted by $\mathbf{a}^\dagger(\mathbf{x})$. They provide a reference for attributing the remaining loss between predictors and delegators, but need not be optimal. The fitting and data-splitting procedure is described in Appendix A.4.

Replacing $\mathbf{a}^*$ by $\mathbf{a}^\dagger$ gives

$$\ell(\hat{\mathbf{y}}, \mathbf{y}) = \underbrace{\ell(F(\mathbf{x}; \mathbf{a}^\dagger), \mathbf{y})}_{\text{predictor loss under restricted-oracle weights}} + \underbrace{[\ell(\hat{\mathbf{y}}, \mathbf{y}) - \ell(F(\mathbf{x}; \mathbf{a}^\dagger), \mathbf{y})]}_{\text{restricted delegator regret}}. \quad (8)$$

The second term measures the difference relative to the restricted oracle and may be negative, since the oracle is not sample-wise optimal. The first term is the predictor loss remaining under that reference. Together, they distinguish potential gains from better delegation from the loss that remains after delegation is improved.

4 Experiments

Our claims are validated empirically over selection of four datasets: CIFAR10 (Krizhevsky, 2009), SVHN (Netzer et al., 2011), Bike sharing dataset (Fanaee-T, 2013), and Energy production dataset (Zaki, 2026). For each dataset, the inputs $\mathbf{x}$ and targets $\mathbf{y}$ were normalized where applicable, and categorical integer-valued features were one-hot encoded. For each random seed, the data were independently partitioned into training and validation sets using an 85%–15% split. Each training run is repeated across 5 different seeds.

For the tabular regression tasks, Bike and Energy, we use fully connected MLPs with two hidden layers, and optimize them using the mean squared error loss, ℓ_{MSE} . The hidden-layer widths decrease progressively toward the output: the two-layer architecture uses widths $(2b, b)$, where b is a model-specific base width. This parameterization preserves a consistent architectural shape while allowing the capacities of the predictor and delegator networks to be varied independently through their respective base widths. Regression tasks are learned for 2000 epochs.

For the image-classification tasks, CIFAR-10 and SVHN, we use three-layer MLPs with hidden widths being $(3b, 2b, b)$, where b is a model-specific base width. Classification tasks are optimized using the cross-entropy loss, ℓ_{CE} . Remaining training hyper-parameters are specified in Table 2. Image tasks are learned for 100 epochs.

Following the load-balancing motivation of Shazeer et al. (2017) and Fedus et al. (2022), we penalize nonuniform predictor usage through normalized Gini impurity. For a minibatch $\mathbb{B} \subseteq \mathbb{D}$, the final training objective is

$$\begin{aligned} \mathcal{L}_{\mathbb{B}} &= \frac{1}{|\mathbb{B}|} \sum_{(\mathbf{x}, \mathbf{y}) \in \mathbb{B}} \ell_{\text{train}}(\mathbf{x}, \mathbf{y}) + \lambda \frac{\sum_{m=1}^M u_m^2 - 1/M}{1 - 1/M}, \\ \mathbf{u} &= \frac{1}{|\mathbb{B}|} \sum_{(\mathbf{x}, \mathbf{y}) \in \mathbb{B}} \bar{\mathbf{a}}(\mathbf{x}), \end{aligned} \quad (9)$$

where $M > 1$, $\lambda = 0.2$, and ℓ_{train} has the value of the ensemble loss but the selected training gradients defined in Section 4.3. This penalty encourages balanced predictor use across the minibatch.

The overall ensemble performance is evaluated on $\mathbb{D}_{\text{val}}$ using the unregularized loss $\ell(\hat{\mathbf{y}}, \mathbf{y})$. We further assess the individual performance and diversity of predictors through $\mathcal{E}_{\text{P}}$ and $\mathcal{A}_{\text{P}}$, and of delegators through $\mathcal{E}_{\text{D}}$ and $\mathcal{A}_{\text{D}}$. Finally, Equation 8 separates the remaining error into potential gains from improved delegation or improved prediction.

4.1 Scaling the number of delegators

We next investigate whether delegation itself benefits from an ensemble effect, and under which model-capacity regimes such benefits emerge (Contribution **(C1)**). In particular, we study whether increasing the number of delegators is most useful when the predictor ensemble is small, the predictors have limited capacity, or the individual delegators are themselves relatively small.

To isolate the effect of the number of delegators, we vary $L \in \{0, 1, 2, 4, 8, 16, 32\}$ while exhaustively evaluating all combinations of the number of predictors $M \in \{2, 4, 8, 16, 32\}$, predictor base width $b_M \in \{4, 8, 16\}$, and delegator base width $b_L \in \{4, 8, 16\}$ across all four tasks. The case $L = 0$ corresponds to fixed uniform delegation weights $\bar{\mathbf{a}}$, whereas $L = 1$ recovers a setting closely related to a standard mixture-of-experts model. Each experimental condition is evaluated over five independent random seeds, yielding 6300 training runs. We use the **Delegators** predictor-ambiguity setting (Section 4.3) with delegator ambiguity disabled (Section 4.4).

4.2 Delegator’s aggregation method

Previous work (Ortega et al., 2022; Wood et al., 2023) characterizes optimal aggregation rules for predictors under a fixed set of weights $\mathbf{a}$. In our setting, however, predictor and delegator parameters are optimized jointly. Consequently, the aggregation rule affects not only the final combination of delegator outputs, but also the optimization dynamics through which predictors and delegators co-adapt. Results derived for fixed-weight aggregation therefore do not directly determine the appropriate aggregation rule for delegators in our model. We therefore evaluate this choice empirically, corresponding to Contribution **(C2)**.

We compare two aggregation rules over the L delegators: the arithmetic mean, $\bar{\mathbf{a}}(\mathbf{x}) = L^{-1} \sum_{l=1}^L \mathbf{a}_l(\mathbf{x})$, and the normalized geometric mean, $\bar{\mathbf{a}}(\mathbf{x}) = Z^{-1} \odot_{l=1}^L \mathbf{a}_l(\mathbf{x})^{L^{-1}}$, where Z normalizes the weights to sum to one. The predictor aggregation in Equation 1 remains unchanged. We use the **None** predictor-ambiguity setting (Section 4.3) with delegator ambiguity enabled (Section 4.4).

To isolate the effect of the aggregation rule, we construct 100 experimental configurations by sampling the remaining experimental variables from the sets reported in Table 3. For each configuration and each of the four tasks, we evaluate a matched pair of experiments differing only in the delegator aggregation rule: arithmetic mean or geometric mean. This yields 800 experimental conditions. Each condition is evaluated over five independent random seeds, for a total of 4,000 training runs.

4.3 Predictor Ambiguity gradient flow

As discussed in Section 2, explicitly maximizing predictor ambiguity can harm individual predictor performance. The standard training procedures suggest only to use the weighted performance $\mathcal{E}_P$, however, the delegators still should identify decorrelated predictors using the predictor ambiguity $\mathcal{A}_P$. We therefore propose to include the ambiguity term $\mathcal{A}_P$ in the loss equation but to stop its gradient from influencing the predictor parameters Θ . We empirically compare three ways to train the predictor ensemble: allow predictor ambiguity to train both predictors and delegators, only delegators, or neither. Table 1 summarizes which terms are used to update each group of parameters. The performance term $\mathcal{E}_P$ always contributes a gradient; the settings differ only in whether ambiguity does.

Table 1: Objectives used to update predictors and delegators when training the delegator ensemble jointly. All terms use the learned aggregate weights $\bar{\mathbf{a}}$. Each column updates only its own parameter group.

Setting	Updates Predictors	Updates delegators
Both	$\mathcal{E}_P - \mathcal{A}_P$	$\mathcal{E}_P - \mathcal{A}_P$
Delegators	$\mathcal{E}_P$	$\mathcal{E}_P - \mathcal{A}_P$
None	$\mathcal{E}_P$	$\mathcal{E}_P$

The **Delegators** setting trains predictors only to be individually accurate. Delegators still account for both accuracy and complementarity when assigning weights to predictors. This retains diversity as a routing criterion without encouraging predictors to become less accurate merely to differ from one another.

These choices change the training gradients but preserve the reported ensemble loss. We implement them with stop-gradient operations; the scalar training objective is given in Appendix A.3. The load-balancing penalty in Equation 9 is applied in every setting.

We compare **Both**, **Delegators**, and **None** with delegator ambiguity enabled and arithmetic delegator aggregation, corresponding to Contribution (C3). Following the protocol of the preceding experiment, we sample 100 configurations from the parameter sets in Table 3 and evaluate all three gradient-flow variants for each configuration and each of the four tasks, holding all other experimental factors fixed. This yields 1,200 experimental conditions. Each condition is evaluated over five independent random seeds, for a total of 6,000 training runs.

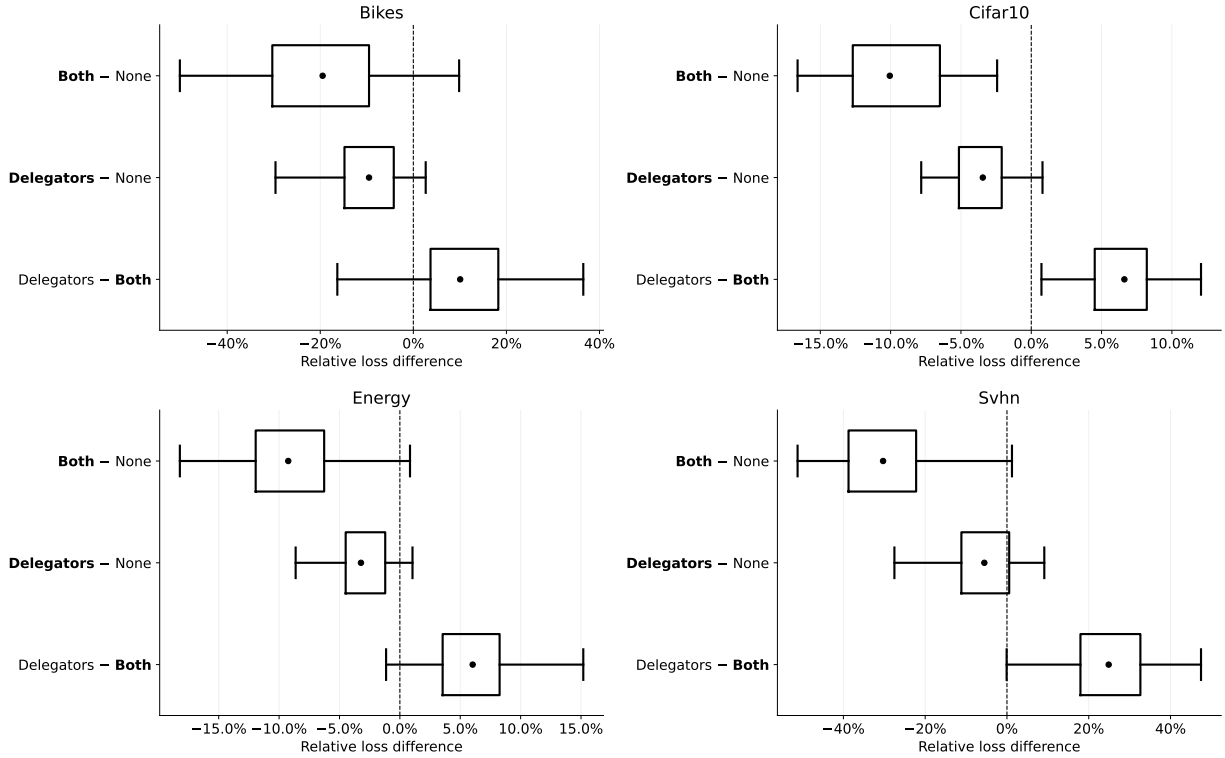


Figure 1: Ambiguity gradient flow: Distribution of the relative differences in validation loss (lower values indicate better performance) between pairs of ambiguity-gradient methods across tasks. The black dot denotes the mean relative difference. For each task and method pair, the superior method is highlighted in bold.

Ensemble performance Figure 1 shows the relative validation performance difference between the ambiguity gradient configurations. In all tasks there is a consistent ranking of the configurations where **Both** > **Delegators** > **None**.

Predictors’ performance

Delegators’ performance

Delegators’ regret analysis We found that in 85% cases the restricted oracle was able to improve the validation loss.

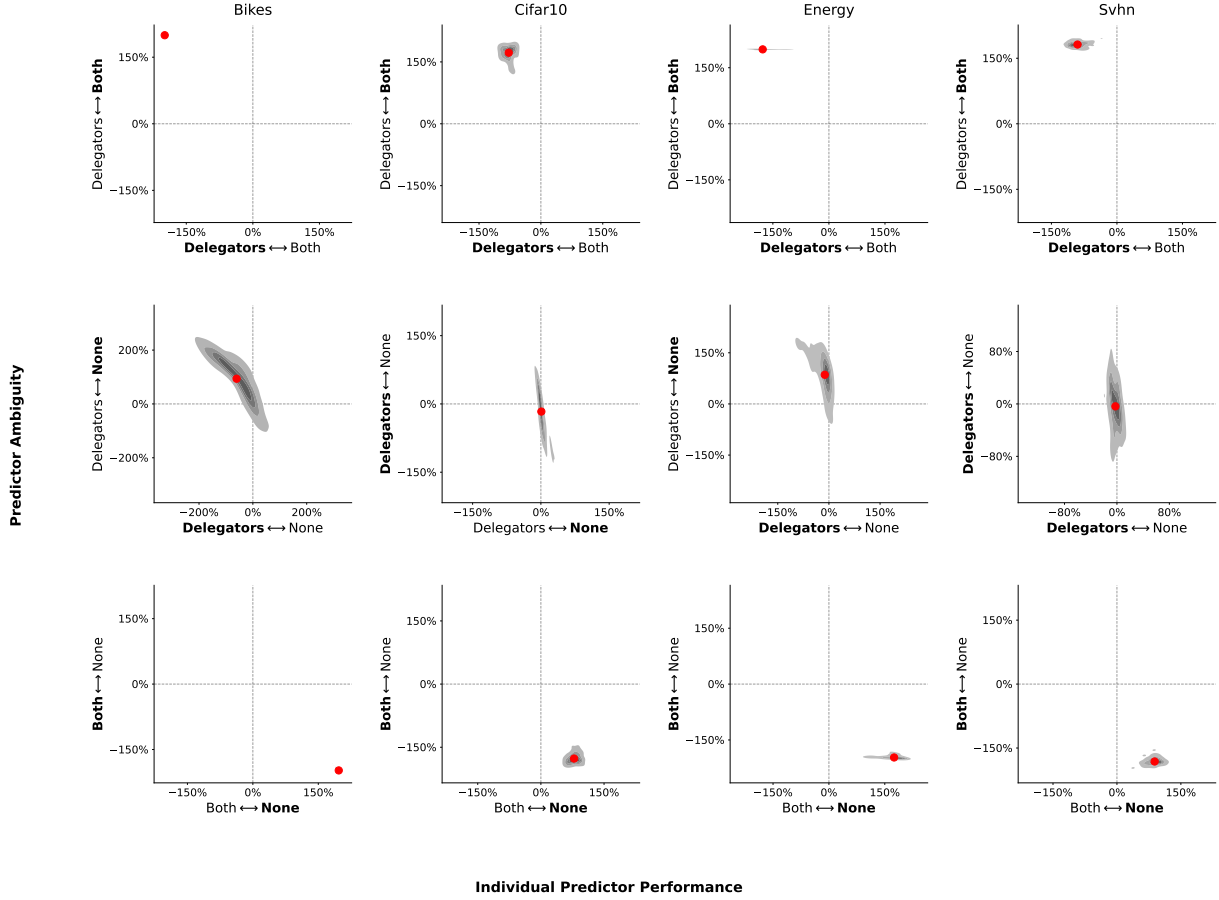


Figure 2: Ambiguity gradient flow: Distribution of the relative differences in weighted predictor performance (x-axis; higher values indicate better performance) and weighted predictor ambiguity (y-axis; higher values indicate greater ambiguity) across tasks and pairs of methods. The red dot denotes the mean of the distribution. For each task and method pair, the superior method is highlighted in bold.

4.4 Delegator Ambiguity gradient flow

We next test whether delegators should be explicitly trained to increase their ambiguity. Joint ensemble training minimizes $\mathcal{E}_D - \mathcal{A}_D$. Blocking the delegator-ambiguity gradient instead trains each delegator to produce an accurate ensemble prediction on its own, relying on different random initializations to induce diversity. We compare two settings, in which delegators receive gradients only from

$$\underbrace{\mathcal{E}_P(\bar{a}) - \mathcal{A}_P(\bar{a})}_{\text{Delegator ambiguity enabled}} \quad \text{or} \quad \underbrace{\mathcal{E}_D}_{\text{Delegator ambiguity disabled}}. \quad (10)$$

The first expression equals $\mathcal{E}_D - \mathcal{A}_D$ and is only valid when predictor ambiguity gradient settings is not **None**. All other task-loss gradients to delegators are stopped; the load-balancing penalty remains active in both settings. The full training equations are annoyingly cumbersome, so we enumerate all combinations of predictor and delegator ambiguity gradient settings, including their stop-gradient operations, in Appendix A.3.

To isolate the effect of delegator ambiguity, we keep the predictor-ambiguity setting fixed at **Delegators** and use arithmetic delegator aggregation. We sample 100 configurations per task from the parameter sets in Table 3. For each configuration, we compare a matched pair with delegator ambiguity enabled or disabled, holding all other factors fixed. Each condition is evaluated over five independent random seeds. Across the four tasks, this yields 800 experimental conditions and 4,000 training runs.

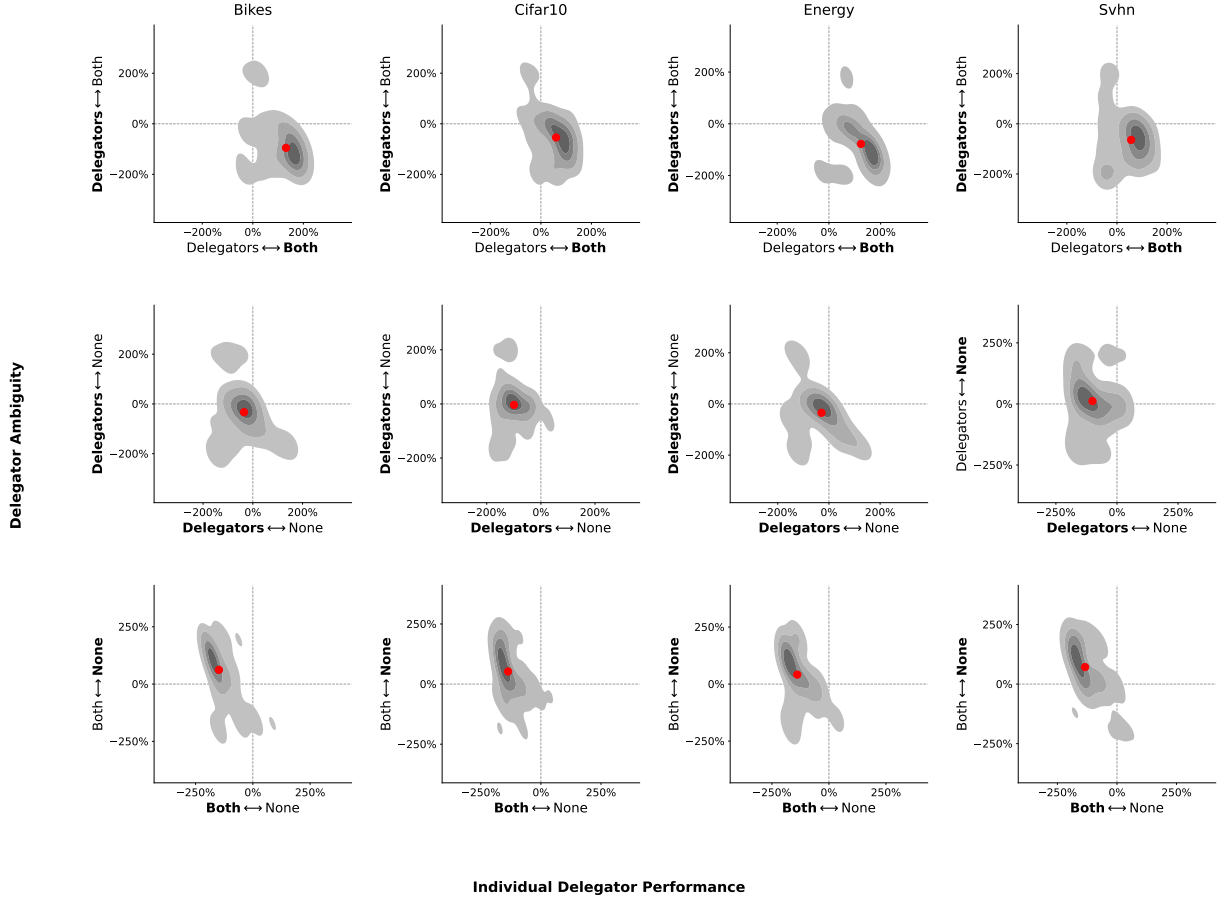


Figure 3: Ambiguity gradient flow: Distribution of the relative differences in individual delegator performance (x-axis; higher values indicate better performance) and delegator ambiguity (y-axis; higher values indicate greater ambiguity) across tasks and pairs of methods. The red dot denotes the distribution mean. For each task and method pair, the superior method is highlighted in bold.

4.5 Analysis

References

- Huanhuan Chen and Xin Yao. Regularized negative correlation learning for neural network ensembles. *IEEE Transactions on Neural Networks*, 20(12):1962–1979, 2009. doi: 10.1109/TNN.2009.2034144.
- Hadi Fanaee-T. Bike Sharing. UCI Machine Learning Repository, 2013. DOI: <https://doi.org/10.24432/C5W894>.
- William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.
- Stanislav Fort, Huiyi Hu, and Balaji Lakshminarayanan. Deep ensembles: A loss landscape perspective. *arXiv preprint arXiv:1912.02757*, 2019.
- Alex Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009.
- Anders Krogh and Jesper Vedelsby. Neural network ensembles, cross validation, and active learning. In G. Tesauro, D. Touretzky, and T. Leen (eds.), *Advances in Neural Information Processing Systems*, vol-

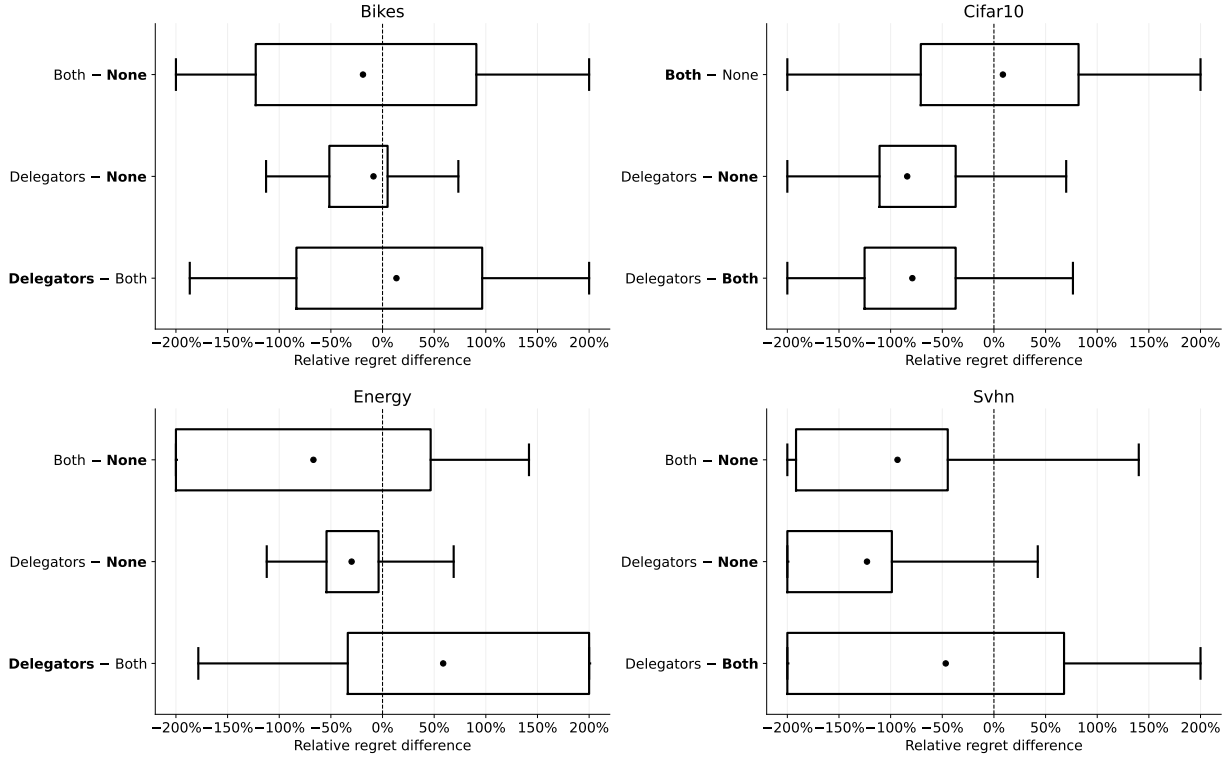


Figure 4: Ambiguity gradient flow: Distribution of the relative differences in delegators’ regret between pairs of ambiguity-gradient methods across tasks. Higher values indicate that a larger fraction of the remaining loss is attributable to the delegators. The black dot denotes the mean relative difference. For each task and method pair, the method with higher delegators’ regret is highlighted in bold.

ume 7. MIT Press, 1994. URL https://proceedings.neurips.cc/paper_files/paper/1994/file/b8c37e33defde51cf91e1e03e51657da-Paper.pdf.

Boan Liu, Liang Ding, Li Shen, Keqin Peng, Yu Cao, Dazhao Cheng, and Dacheng Tao. Diversifying the mixture-of-experts representation for language models with orthogonal optimizer. In *ECAI 2024: 27th European Conference on Artificial Intelligence, 19–24 October 2024, Santiago de Compostela, Spain—Including 13th Conference on Prestigious Applications of Intelligent Systems (PAIS 2024)*, pp. 2966–2973. SAGE Publications Pvt. Ltd 1 Oliver’s Yard, 55 City Road, London, EC1Y 1SP, 2024.

Yuval Netzer, Tao Wang, Adam Coates, Alessandro Bissacco, Baolin Wu, Andrew Y Ng, et al. Reading digits in natural images with unsupervised feature learning. In *NIPS workshop on deep learning and unsupervised feature learning*, volume 2011, pp. 4. Granada, 2011.

Luis A. Ortega, Rafael Cabañas, and Andres Masegosa. Diversity and generalization in neural network ensembles. In Gustau Camps-Valls, Francisco J. R. Ruiz, and Isabel Valera (eds.), *Proceedings of The 25th International Conference on Artificial Intelligence and Statistics*, volume 151 of *Proceedings of Machine Learning Research*, pp. 11720–11743. PMLR, 28–30 Mar 2022. URL <https://proceedings.mlr.press/v151/ortega22a.html>.

Junfeng Ran, Guangxiang Zhao, Yuhang Wu, Dawei Zhu, Longyun Wu, Yikai Zhao, Tong Yang, Lin Sun, Xiangzheng Zhang, and Sujian Li. Router upcycling: Leveraging mixture-of-routers in mixture-of-experts upcycling. *arXiv preprint arXiv:2509.00679*, 2025.

Table 2: Training and initialization hyperparameters used across experiments.

HYPERPARAMETER	VALUE
Kernel initialization	LeCun normal
Bias initialization	Zero
Hidden-layer activation	ReLU
Optimizer	Adam
Learning rate	10^{-3}
Weight-decay coefficient	10^{-4}
Tabular-task batch size	256
Image-task batch size	64
Tabular-task training epochs	2000
Image-task training epochs	50

Table 3: Experimental values and their permitted set.

VARIABLE	SET	NOTES
Number of predictors (M)	$\{2, 4, 8, 16, 32\}$	
Number of delegators (L)	$\{0, 1, 2, 4, 8, 16, 32\}$	$L = 0$: uniform weights; $L = 1$: MoE.
Predictors’ base width (b_M)	$\{4, 8, 16\}$	Predictor hidden layers are either $(2b, b)$ or $(3b, 2b, b)$.
Delegators’ base width (b_L)	$\{4, 8, 16\}$	Delegator hidden layers are either $(2b, b)$ or $(3b, 2b, b)$.
Delegators’ aggregation method	$\{\text{mean, geometric mean}\}$	See Section 4.2
Predictor ambiguity gradients	$\{\text{none, delegators, both}\}$	See Table 1.
Delegator ambiguity gradient	$\{\text{disabled, enabled}\}$	Section 4.4.

Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarczyk, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538*, 2017.

Ziteng Wang, Jun Zhu, and Jianfei Chen. Remoe: Fully differentiable mixture-of-experts with relu routing. In *International Conference on Learning Representations*, volume 2025, pp. 59486–59507, 2025.

Danny Wood, Tingting Mu, Andrew M Webb, Henry WJ Reeve, Mikel Lujan, and Gavin Brown. A unified theory of diversity in ensemble learning. *Journal of machine learning research*, 24(359):1–49, 2023.

Ahmed Mohamed Zaki. Wind and solar energy production dataset. Kaggle, 2026. URL <https://www.kaggle.com/datasets/ahmeduzaki/wind-and-solar-energy-production-dataset/code>.

Jia-Chen Zhang, Yu-Jie Xiong, Xi-He Qiu, Chun-Ming Xia, Fei Dai, and Zheng Zhou. Mixture of routers. *arXiv preprint arXiv:2503.23362*, 2025.

A Appendix

A.1 Training specifications

A.2 Delegator performance–ambiguity decomposition under the sum rule

Fix a sample $(\mathbf{x}, \mathbf{y})$ and $L \geq 1$ delegators. Under the sum rule, their weight proposals are averaged:

$$\bar{\mathbf{a}} = \frac{1}{L} \sum_{l=1}^L \mathbf{a}_l, \quad \hat{\mathbf{y}}^{(l)} = F(\mathbf{x}; \mathbf{a}_l), \quad \hat{\mathbf{y}} = F(\mathbf{x}; \bar{\mathbf{a}}). \quad (11)$$

We show that this rule makes the final prediction the appropriate aggregate of the delegator-induced predictions, and then derive $\ell(\hat{\mathbf{y}}, \mathbf{y}) = \mathcal{E}_D - \mathcal{A}_D$.

Regression. Predictor outputs are combined by a weighted arithmetic mean. Writing $a_{l,m}$ for the weight assigned to predictor m by delegator l , we obtain

$$\begin{aligned}\hat{\mathbf{y}} &= \sum_{m=1}^M \bar{a}_m \hat{\mathbf{y}}_m = \frac{1}{L} \sum_{l=1}^L \sum_{m=1}^M a_{l,m} \hat{\mathbf{y}}_m = \frac{1}{L} \sum_{l=1}^L \hat{\mathbf{y}}^{(l)} \\ \mathbf{0} &= \frac{1}{L} \sum_{l=1}^L \hat{\mathbf{y}}^{(l)} - \hat{\mathbf{y}} = \sum_{l=1}^L (\hat{\mathbf{y}}^{(l)} - \hat{\mathbf{y}})\end{aligned}\tag{12}$$

Expanding each individual squared error around $\hat{\mathbf{y}}$ gives

$$\begin{aligned}\mathcal{E}_D &= \frac{1}{Ld_y} \sum_{l=1}^L \left\| (\hat{\mathbf{y}}^{(l)} - \hat{\mathbf{y}}) + (\hat{\mathbf{y}} - \mathbf{y}) \right\|_2^2 \\ &= \frac{1}{Ld_y} \sum_{l=1}^L \left\| \hat{\mathbf{y}}^{(l)} - \hat{\mathbf{y}} \right\|_2^2 + \frac{2}{Ld_y} (\hat{\mathbf{y}} - \mathbf{y})^\top \underbrace{\sum_{l=1}^L (\hat{\mathbf{y}}^{(l)} - \hat{\mathbf{y}})}_{\mathbf{0}} + \frac{1}{Ld_y} \sum_{l=1}^L \left\| \hat{\mathbf{y}} - \mathbf{y} \right\|_2^2 \\ &= \mathcal{A}_D + SE(\hat{\mathbf{y}}, \mathbf{y})\end{aligned}\tag{13}$$

The cross term vanishes, yielding

$$SE(\hat{\mathbf{y}}, \mathbf{y}) = \mathcal{E}_D - \mathcal{A}_D.\tag{14}$$

Classification. Predictor outputs are combined by a normalized weighted geometric mean. Assume positive predictor probabilities, so the logarithms below are defined. For class c , the prediction induced by delegator l is

$$\hat{y}_c^{(l)} = \frac{1}{Z_l} \prod_{m=1}^M \hat{y}_{m,c}^{a_{l,m}}, \quad Z_l = \sum_{c=1}^{d_y} \prod_{m=1}^M \hat{y}_{m,c}^{a_{l,m}}.\tag{15}$$

Taking the geometric mean across delegators gives

$$\prod_{l=1}^L (\hat{y}_c^{(l)})^{1/L} = \left(\prod_{l=1}^L Z_l^{-1/L} \right) \prod_{m=1}^M \hat{y}_{m,c}^{\frac{1}{L} \sum_{l=1}^L a_{l,m}} = \left(\prod_{l=1}^L Z_l^{-1/L} \right) \prod_{m=1}^M \hat{y}_{m,c}^{\bar{a}_m}.\tag{16}$$

The factor involving Z_l is the same for every class and cancels when the probabilities are normalized. Hence the final prediction is the normalized geometric mean of the induced predictions:

$$\hat{y}_c = \frac{1}{Z_D} \prod_{l=1}^L (\hat{y}_c^{(l)})^{1/L}, \quad Z_D = \sum_{c=1}^{d_y} \prod_{l=1}^L (\hat{y}_c^{(l)})^{1/L}.\tag{17}$$

Taking logarithms yields

$$\frac{1}{L} \sum_{l=1}^L \log \hat{y}_c^{(l)} = \log \hat{y}_c + \log Z_D.\tag{18}$$

Using $\sum_c y_c = 1$, the mean individual delegator loss is therefore

$$\begin{aligned}\mathcal{E}_D &= - \sum_{c=1}^{d_y} y_c \left(\frac{1}{L} \sum_{l=1}^L \log \hat{y}_c^{(l)} \right) \\ &= CE(\hat{\mathbf{y}}, \mathbf{y}) - \log Z_D.\end{aligned}\tag{19}$$

Similarly, using $\sum_c \hat{y}_c = 1$, the mean delegator ambiguity is

$$\begin{aligned}\mathcal{A}_D &= \frac{1}{L} \sum_{l=1}^L \text{KL}(\hat{\mathbf{y}} \parallel \hat{\mathbf{y}}^{(l)}) \\ &= \sum_{c=1}^{d_y} \hat{y}_c \left(\log \hat{y}_c - \frac{1}{L} \sum_{l=1}^L \log \hat{y}_c^{(l)} \right) = -\log Z_D.\end{aligned}\tag{20}$$

Subtracting the two expressions gives

$$\text{CE}(\hat{\mathbf{y}}, \mathbf{y}) = \mathcal{E}_D - \mathcal{A}_D.\tag{21}$$

Equations 14 and 21 establish Equation 4. Both rely on arithmetic aggregation of delegation weights; geometric weight aggregation does not generally produce the required mean of the induced predictions.

A.3 Gradient routing

We enumerate all combinations of predictor ambiguity gradient (PAG) and delegator ambiguity gradient (DAG) settings. These abbreviations are used only in this appendix. We assume arithmetic delegation aggregation and $L \geq 1$, so the ensemble loss satisfies

$$\ell = \mathcal{E}_P(\bar{\mathbf{a}}) - \mathcal{A}_P(\bar{\mathbf{a}}) = \mathcal{E}_D - \mathcal{A}_D, \quad \mathcal{E}_D = \frac{1}{L} \sum_{l=1}^L [\mathcal{E}_P(\mathbf{a}_l) - \mathcal{A}_P(\mathbf{a}_l)].\tag{22}$$

The operator sg_{Θ} preserves its argument's value but stops its gradient with respect to predictor parameters. Similarly, sg_{Φ} stops gradients with respect to delegator parameters, while sg stops all gradients. The expressions below define ℓ_{train} : each has the same numerical value as ℓ , but supplies the gradients prescribed by its setting.

All settings also include the auxiliary losses, including the load-balancing penalty in Equation 9. These are added unchanged and are not subject to the stop-gradient operations below.

PAG = Both, DAG = True Both predictors and delegators receive the full ensemble-loss gradient:

$$\ell_{\text{train}} = \ell.\tag{23}$$

PAG = Delegators, DAG = True Predictor ambiguity updates only delegators:

$$\ell_{\text{train}} = \mathcal{E}_P(\bar{\mathbf{a}}) - \text{sg}_{\Theta} [\mathcal{A}_P(\bar{\mathbf{a}})].\tag{24}$$

PAG = None, DAG = True Predictor ambiguity contributes no gradient:

$$\ell_{\text{train}} = \mathcal{E}_P(\bar{\mathbf{a}}) - \text{sg} [\mathcal{A}_P(\bar{\mathbf{a}})].\tag{25}$$

PAG = Both, DAG = False Predictors receive the full ensemble-loss gradient, while delegators learn only from their individual performance:

$$\begin{aligned}\ell_{\text{train}} &= \text{sg}_{\Phi} [\ell] \\ &\quad + \text{sg}_{\Theta} [\mathcal{E}_D - \text{sg}_{\Phi} [\mathcal{A}_D] + \text{sg}_{\Phi} [\mathcal{A}_D - \mathcal{E}_D]].\end{aligned}\tag{26}$$

PAG = Delegators, DAG = False Predictors learn only from their weighted individual performance. Delegators learn from their individual performance, which still accounts for predictor ambiguity:

$$\begin{aligned}\ell_{\text{train}} &= \text{sg}_{\Phi} [\mathcal{E}_P(\bar{\mathbf{a}}) - \text{sg}_{\Theta} [\mathcal{A}_P(\bar{\mathbf{a}})]] \\ &\quad + \text{sg}_{\Theta} [\mathcal{E}_D - \text{sg}_{\Phi} [\mathcal{A}_D] + \text{sg}_{\Phi} [\mathcal{A}_D - \mathcal{E}_D]].\end{aligned}\tag{27}$$

PAG = None, DAG = False Predictors learn only from their weighted individual performance. Delegators also ignore predictor ambiguity within each individual weight proposal:

$$\begin{aligned} \ell_{\text{train}} = & \text{sg}_{\Phi} [\mathcal{E}_{\text{P}}(\bar{\mathbf{a}}) - \text{sg} [\mathcal{A}_{\text{P}}(\bar{\mathbf{a}})]] \\ & + \text{sg}_{\Theta} \left[\frac{1}{L} \sum_{l=1}^L (\mathcal{E}_{\text{P}}(\mathbf{a}_l) - \text{sg}_{\Phi} [\mathcal{A}_{\text{P}}(\mathbf{a}_l)]) - \text{sg}_{\Phi} [\mathcal{A}_{\text{D}}] + \text{sg}_{\Phi} [\mathcal{A}_{\text{D}} - \mathcal{E}_{\text{D}}] \right]. \end{aligned} \quad (28)$$

In the DAG = False settings, the second bracket has numerical value zero but supplies the required delegator gradients. The correction term $\text{sg}_{\Phi} [\mathcal{A}_{\text{D}} - \mathcal{E}_{\text{D}}]$ therefore preserves the reported ensemble loss without changing either parameter group’s updates.

A.4 Restricted delegator oracle training procedure

For candidate delegator parameters Φ' , let $\bar{\mathbf{a}}_{\Phi'}(\mathbf{x})$ denote their aggregate weights. With predictors fixed, the oracle minimizes

$$\mathcal{L}_{\text{oracle}}(\Phi') = \frac{1}{|\mathbb{D}_{\text{fit}}|} \sum_{(\mathbf{x}, \mathbf{y}) \in \mathbb{D}_{\text{fit}}} \ell(F(\mathbf{x}; \bar{\mathbf{a}}_{\Phi'}(\mathbf{x})), \mathbf{y}). \quad (29)$$

For each trained seed, we freeze all predictor parameters and cache the validation predictions and each delegator’s penultimate-layer features. Starting from the learned delegators, we fit only the kernels and biases of their final affine layers using full-batch L-BFGS with history size 10. The objective is Equation 29 with $\mathbb{D}_{\text{fit}} = \mathbb{D}_{\text{val}}$, without a routing tether, load balancing, weight decay, or ambiguity-gradient restrictions. We perform at most 100 updates and stop when the gradient norm is at most 10^{-6} or a nonfinite value is encountered. Each seed retains the weights with the lowest validation loss, including the unmodified initialization. The softmax routing makes this optimization generally nonconvex, so the resulting weights are a restricted reference rather than a certified optimum. Because validation labels participate in fitting and selection, this procedure quantifies recoverable loss on that dataset only.